The Companion Explainer

Understanding the Math Behind Raman Suppression via Spectral Phase Optimization

A First-Principles Walkthrough for Undergrads

Rivera Lab, Cornell University

Companion to the Verification Document

Prepared for Ignacio Rivera

April 2026

Abstract

This document explains every piece of math in the Raman suppression optimization project, starting from Maxwell's equations and building up to the adjoint method and the log-scale cost function. It's written for an undergrad physics major who knows Fourier transforms, wave equations, and complex numbers — and wants to understand *why* each equation is the way it is, not just *what* it says.

How to read this: Parts 1–3 give you the physics and the optimization setup. Part 4 is the key insight (the adjoint method). Parts 5–8 cover the tricks that make it work in practice. Parts 9–10 show how we verify correctness. Parts 11–13 go deeper: the ML connection, the GNLSE from Maxwell's equations, and the adjoint from the Lagrangian.

Prerequisite: Fourier transforms, basic complex numbers, the wave equation. That's it.

Contents

1 The Big Picture (No Math Yet) **4**

2 The Wave Equation (The GNLSE) **5**
- 2.1 From Maxwell's Equations to One Equation . . . 5
- 2.2 What Each Term Means . . . 5
- 2.3 The Interaction Picture . . . 6

3 The Optimization Problem **6**
- 3.1 What We Control . . . 6
- 3.2 What We Minimize . . . 6
- 3.3 The Landscape . . . 7

4 How Optimization Works: From Gradient Descent to L-BFGS 7
4.1 Gradient Descent: The Simplest Method . . . 7
4.2 Newton's Method: Using Curvature Information . . . 8
4.3 Quasi-Newton: Approximate the Hessian from Gradient History . . . 8
4.4 BFGS: The Best Quasi-Newton Update . . . 9
4.5 L-BFGS: The "Limited Memory" Version We Actually Use . . . 9
4.6 The Line Search: Picking the Step Size . . . 10
4.7 Why L-BFGS Is Perfect for Our Problem . . . 11

5 The Adjoint Method (The Key Insight) 11
5.1 Why Brute Force Fails . . . 11
5.2 The Adjoint Trick: Run the Movie Backwards . . . 12
5.3 The Relay Race Analogy . . . 12
5.4 The Adjoint Equation . . . 12
5.5 The Chain Rule . . . 13

6 The Log-Scale Trick 13
6.1 The Vanishing Gradient Problem . . . 13
6.2 The Fix: Optimize in Decibels . . . 13

7 The Soliton Number 14

8 Wirtinger Derivatives Demystified 15
8.1 The Problem . . . 15
8.2 Wirtinger's Solution . . . 15
8.3 The Factor of 2 . . . 16

9 Why FFT Factors of N_t Appear 16

10 Verification 16
10.1 Taylor Remainder Test . . . 16
10.2 Finite Difference Check . . . 16

11 What's Left in the Full Verification Document 16

12 The ML Connection 17
12.1 Forward Pass: Layers vs. Fiber Steps . . . 17
12.2 Backprop = Adjoint . . . 17
12.3 Why L-BFGS Instead of Adam? . . . 17
12.4 Log-Cost = Focal Loss . . . 18

13 First Principles: Where the GNLSE Comes From 18
13.1 Maxwell's Equations . . . 18
13.2 Slowly-Varying Envelope Approximation . . . 18
13.3 Adding Raman . . . 18
13.4 Where γ and $h_R(t)$ Come From . . . 19

14 The Adjoint from First Principles 19
14.1 Constrained Optimization via the Lagrangian . . . 19

15 Results: What Does Suppression Look Like? 19

16 What the April 2026 Parallel-Session Push Actually Showed 23

16.1 The suppression record, re-baselined 23
16.2 The optimum is on a razor's edge 23
16.3 The simple phase is a good *initialiser*, not a universal minimum 24
16.4 Determinism 24
16.5 What did *not* survive 24

1 The Big Picture (No Math Yet)

Imagine a laser pulse — a burst of light lasting 185 femtoseconds (185×10^{-15} s) — traveling through an optical fiber. The fiber is glass (SiO_2), and the glass has atoms that vibrate. When the intense pulse hits these atoms, some of the light's energy excites the vibrations and gets re-emitted at a lower frequency. This is **Raman scattering**: light loses energy and shifts to redder colors.

Three things happen simultaneously as the pulse propagates:

1. **Dispersion:** Different colors travel at different speeds. The pulse spreads out.
2. **Kerr effect:** The light is so intense it changes the glass's refractive index. The pulse modifies its own phase.
3. **Raman scattering:** The glass steals energy and red-shifts it.

Our approach: Before the pulse enters the fiber, we reshape its *spectral phase* using a pulse shaper — a device that delays different colors by different amounts. By choosing the right delay pattern, we make the pulse propagate through the fiber in a way that minimizes Raman scattering.

Intuition

We can only control the *timing* of different colors, not which colors are present. It's like adjusting when each runner starts in a relay race, but not which runners participate. The timing determines the temporal shape of the pulse, which determines how it interacts with the glass.

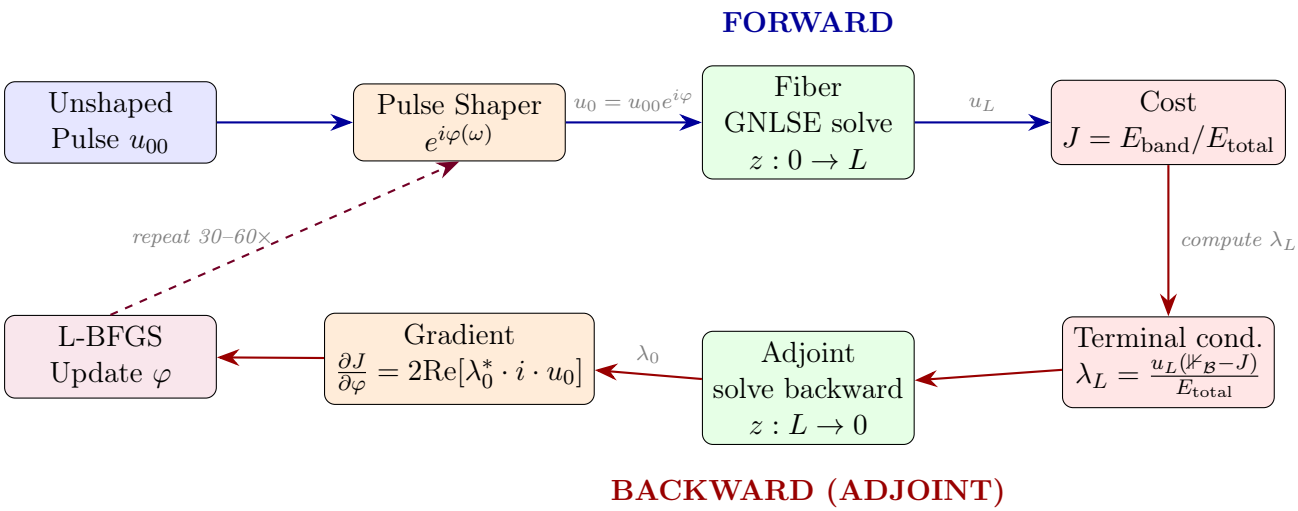

Figure 1: The optimization pipeline. Blue arrows: forward propagation computes the cost. Red arrows: adjoint propagation computes the gradient. Purple dashed: L-BFGS updates the phase and repeats. One full cycle takes ∼2 seconds.

2 The Wave Equation (The GNLSE)

2.1 From Maxwell's Equations to One Equation

You've seen the wave equation:

$$\frac{\partial^2 E}{\partial z^2} = \frac{1}{c^2}\frac{\partial^2 E}{\partial t^2} \tag{1}$$

For light in a fiber, we need three modifications: frequency-dependent speed (dispersion), intensity-dependent refractive index (Kerr), and energy transfer between frequencies (Raman). The result is the **Generalized Nonlinear Schrödinger Equation (GNLSE)**:

Key Equation

$$\frac{\partial u}{\partial z} = \underbrace{iD(\omega)\cdot u}_{\text{dispersion}} + \underbrace{i\gamma(1-f_R)|u|^2 u}_{\text{Kerr}} + \underbrace{i\gamma f_R\,(h_R * |u|^2)\,u}_{\text{Raman}} \tag{2}$$

2.2 What Each Term Means

The field $u(z,t)$ is the complex envelope of the electric field. $|u|^2$ gives instantaneous power in Watts. It depends on position z along the fiber and time t.

Dispersion $iD(\omega)u$**:** Each color ω picks up phase $D(\omega)\cdot z$ per meter:

$$D(\omega) = \frac{\beta_2}{2}\omega^2 + \frac{\beta_3}{6}\omega^3 + \cdots \tag{3}$$

- β_2 = group velocity dispersion. Negative $\Rightarrow$ anomalous dispersion (longer wavelengths faster).
- β_3 = third-order dispersion. Usually a small correction.

Think of $D(\omega)$ as a recipe: "delay blue by this much, delay red by that much, per meter of fiber."

Kerr effect $i\gamma|u|^2u$**:** The refractive index changes as $n = n_0 + n_2|E|^2$. The pulse phase shifts proportionally to its own intensity. The constant γ (gamma) measures the fiber's nonlinear strength:

SMF-28:	$\gamma = 0.0011$ W^{-1}m^{-1}	(weakly nonlinear)
HNLF:	$\gamma = 0.010$ W^{-1}m^{-1}	(10× stronger)

Raman effect $i\gamma f_R(h_R * |u|^2)u$**:** The glass responds with a delay — atoms take ∼30 fs to ring up. The response function $h_R(t)$ is a damped oscillation:

$$h_R(t) = \frac{\tau_1^2+\tau_2^2}{\tau_1\tau_2^2}\, e^{-t/\tau_2}\sin(t/\tau_1)\cdot\Theta(t) \tag{4}$$

where $\tau_1 = 12.2$ fs (oscillation), $\tau_2 = 32$ fs (decay), and $\Theta(t)$ is the Heaviside function (causality: glass responds *after* being hit). The $*$ means convolution — the Raman response depends on the intensity history over the previous ∼100 fs. The fraction $f_R = 0.18$ means 18% Raman (delayed), 82% Kerr (instantaneous).

2.3 The Interaction Picture

Intuition

The dispersion term causes rapid oscillations that waste the ODE solver's time. It's like simulating a vibrating guitar string by tracking every oscillation when you only care about the slowly-changing envelope. The fix: go to a "rotating frame" (the interaction picture) where dispersion vanishes.

Define a new variable that "rotates with" the dispersion:

$$\tilde{u}_\omega(z) = e^{-iD(\omega)z} \cdot u_\omega(z) \tag{5}$$

This is exactly like a rotating reference frame in classical mechanics. In this frame, only the nonlinear terms remain:

$$\frac{d\tilde{u}_\omega}{dz} = i \cdot e^{-iDz} \cdot [\text{nonlinear terms in the lab frame}] \cdot e^{+iDz} \tag{6}$$

The ODE solver can now take large steps. To recover the physical field: $u_\omega = e^{+iDz}\tilde{u}_\omega$.

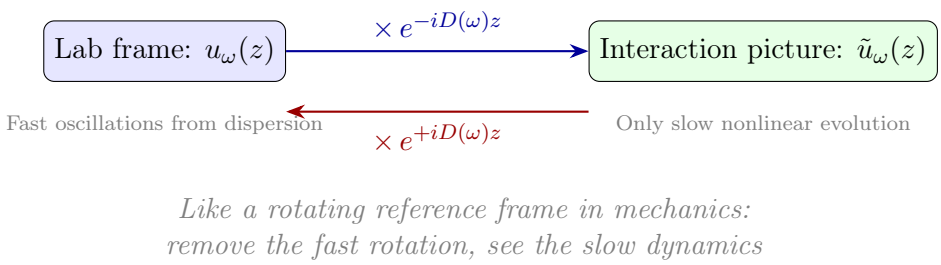

Figure 2: The interaction picture removes fast dispersive oscillations, letting the ODE solver take larger steps. Transform to lab frame when you need the physical field.

3 The Optimization Problem

3.1 What We Control

A pulse shaper applies a spectral phase $\varphi(\omega)$:

$$u_0(\omega) = u_{00}(\omega) \cdot e^{i\varphi(\omega)} \tag{7}$$

where u_{00} is the original pulse. This changes the temporal shape without changing the power spectrum ($|u_0|^2 = |u_{00}|^2$).

3.2 What We Minimize

After propagation, measure the fractional energy in the Raman band:

Key Equation

$$J[\varphi] = \frac{\sum_{\omega \in \mathcal{B}} |u_L(\omega)|^2}{\sum_{\omega} |u_L(\omega)|^2} = \frac{E_{\text{band}}}{E_{\text{total}}} \tag{8}$$

where $\mathcal{B}$ = frequencies shifted > 5 THz red of pump. $J = 0$ is perfect suppression.

In decibels: $J_{\text{dB}} = 10 \log_{10}(J)$. Our best: -78 dB $= 1.6 \times 10^{-8}$.

3.3 The Landscape

J depends on $\varphi(\omega)$ — a vector with $N_t = 8192$ components. We need to find the φ that minimizes J in this 8192-dimensional space. For that, we need the **gradient**: a vector pointing in the direction of steepest descent.

4 How Optimization Works: From Gradient Descent to L-BFGS

This section builds up the optimization algorithm from first principles. If you've taken multivariable calculus, you already know the core ideas.

4.1 Gradient Descent: The Simplest Method

You're standing on a hilly landscape in fog. You can't see far, but you can feel the slope under your feet. The obvious strategy: take a step in the downhill direction.

Mathematically, if φ is your current position (the spectral phase vector) and ∇J is the gradient (the "slope" in each direction), then:

$$\varphi_{\text{new}} = \varphi_{\text{old}} - \alpha \nabla J \tag{9}$$

where $\alpha > 0$ is the **step size** (or "learning rate" in ML). This is gradient descent.

Watch Out

The step size problem. If α is too large, you overshoot the minimum and bounce around. If too small, convergence takes forever. Worse: different directions may need different step sizes. Imagine a long, narrow valley — you need large steps along the valley floor but tiny steps perpendicular to it. Gradient descent with a single α zig-zags, wasting most of its effort.

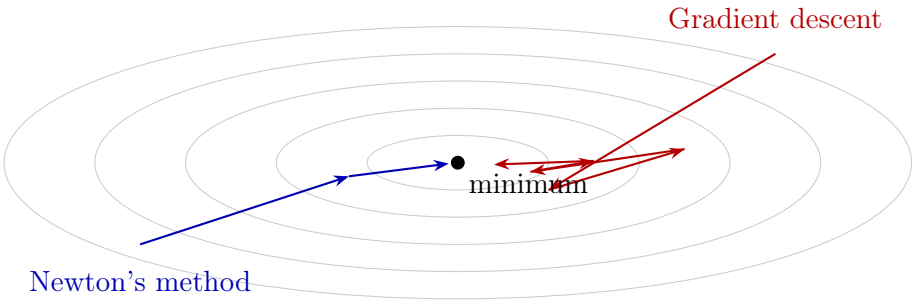

Figure 3: In an elongated valley, gradient descent (red) zig-zags because the gradient always points perpendicular to the contours, not toward the minimum. Newton's method (blue) accounts for curvature and heads straight for the minimum.

4.2 Newton's Method: Using Curvature Information

The fix for zig-zagging: use **curvature** (second-derivative information) to adapt the step size in each direction automatically. Taylor-expand J to second order around the current φ:

$$J(\varphi + \delta\varphi) \approx J(\varphi) + \nabla J \cdot \delta\varphi + \frac{1}{2}\,\delta\varphi^T \cdot H \cdot \delta\varphi \tag{10}$$

where $H = \nabla^2 J$ is the **Hessian matrix** (the matrix of second derivatives, $H_{ij} = \partial^2 J/\partial\varphi_i \partial\varphi_j$).

Setting the derivative of this quadratic to zero gives the Newton step:

$$\delta\varphi = -H^{-1}\nabla J \tag{11}$$

Intuition

Compare to gradient descent (Eq. 9): Newton replaces the scalar step size α with the *inverse Hessian* H^{-1}. This is a matrix that automatically scales the step differently in each direction. In the narrow valley direction, H has a large eigenvalue (high curvature), so H^{-1} gives a small step. In the long valley direction, H has a small eigenvalue, so H^{-1} gives a large step. Result: you walk straight to the minimum.

The problem: H is an $N_t \times N_t$ matrix. With $N_t = 8192$: H has 67 million entries. Computing H requires N_t gradient evaluations. Inverting it costs $O(N_t^3)$. For $N_t = 8192$, this is completely infeasible.

4.3 Quasi-Newton: Approximate the Hessian from Gradient History

Key insight: we don't need the exact Hessian. We can *approximate* it by observing how the gradient changes from step to step.

If the gradient changed by $\Delta g = \nabla J_{\text{new}} - \nabla J_{\text{old}}$ when we moved by $\Delta\varphi = \varphi_{\text{new}} - \varphi_{\text{old}}$, then by definition:

$$H \cdot \Delta\varphi \approx \Delta g \qquad \text{(the "secant condition")} \tag{12}$$

This gives us information about H without computing it directly. After several steps, we accumulate enough $(\Delta\varphi, \Delta g)$ pairs to build a decent approximation.

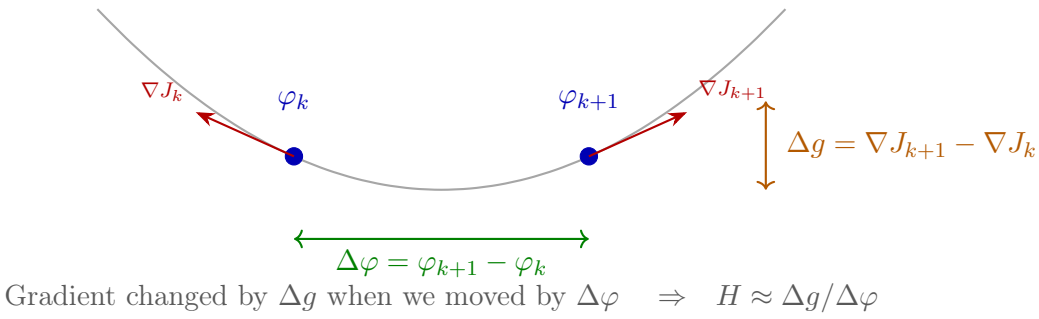

Figure 4: The quasi-Newton idea: observe how the gradient changes between steps. The ratio $\Delta g/\Delta\varphi$ gives information about the Hessian (curvature) without computing it directly.

4.4 BFGS: The Best Quasi-Newton Update

The BFGS update (Broyden–Fletcher–Goldfarb–Shanno, 1970) is a specific formula for updating the Hessian approximation B_k at each step using the secant condition. It maintains two properties:

1. B_k is symmetric positive-definite (so the search direction is always downhill)
2. B_k satisfies the secant condition $B_k \Delta\varphi = \Delta g$ for the most recent step

The update formula (you don't need to memorize this):

$$B_{k+1} = B_k - \frac{B_k \, \Delta\varphi \, \Delta\varphi^T B_k}{\Delta\varphi^T B_k \, \Delta\varphi} + \frac{\Delta g \, \Delta g^T}{\Delta g^T \Delta\varphi} \tag{13}$$

In practice, we work with the *inverse* Hessian approximation B_k^{-1} directly (to avoid solving a linear system at each step).

4.5 L-BFGS: The "Limited Memory" Version We Actually Use

Problem with BFGS: Storing B_k^{-1} requires N_t^2 memory. For $N_t = 8192$: 512 MB per matrix. Not terrible, but wasteful.

L-BFGS solution: Don't store B_k^{-1} explicitly. Instead, store only the last m pairs $\{(\Delta\varphi_i, \Delta g_i)\}_{i=k-m}^{k-1}$ (typically $m = 10$). When you need $B_k^{-1}\nabla J$ (the search direction), compute it on-the-fly using a clever two-loop recursion that only touches the stored pairs.

Memory: $O(m \cdot N_t)$ instead of $O(N_t^2)$. For $m = 10$, $N_t = 8192$: 640 KB vs. 512 MB.

Cost per iteration: One gradient evaluation (forward + adjoint ≈ 2 s) plus $O(m \cdot N_t)$ linear algebra (≈ 0.001 s). The gradient dominates.

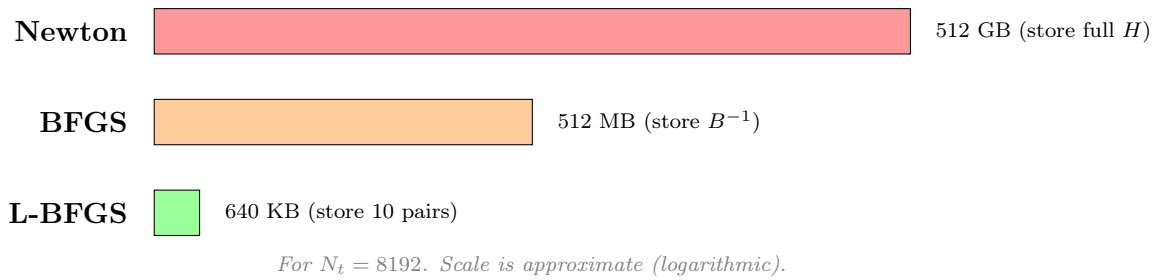

Figure 5: Memory comparison for $N_t = 8192$. Newton stores the full Hessian. BFGS stores an approximation. L-BFGS stores only 10 vector pairs — nearly 1000× less memory than BFGS.

Key Equation

The L-BFGS algorithm:

1. Start with initial phase φ_0 (usually zeros)
2. Compute gradient ∇J via forward + adjoint solve (∼2 s)
3. Compute search direction $d = -B_k^{-1}\nabla J$ using stored $(\Delta\varphi, \Delta g)$ pairs
4. **Line search:** Find step size α satisfying the Wolfe conditions (sufficient decrease + curvature condition). This requires 1–3 additional forward evaluations.
5. Update: $\varphi_{k+1} = \varphi_k + \alpha\, d$
6. Store $(\Delta\varphi, \Delta g)$ pair; discard oldest if $> m$ pairs stored
7. Repeat until $|\nabla J| <$ tolerance or iteration limit reached

4.6 The Line Search: Picking the Step Size

Even with a good direction d, we need to choose how far to step. Too far → overshoot. Too short → wasted iteration.

The **Wolfe conditions** define an acceptable step size α:

$$\text{Sufficient decrease:} \quad J(\varphi + \alpha d) \leq J(\varphi) + c_1\, \alpha\, \nabla J \cdot d \tag{14}$$

$$\text{Curvature condition:} \quad \nabla J(\varphi + \alpha d) \cdot d \geq c_2\, \nabla J \cdot d \tag{15}$$

with $0 < c_1 < c_2 < 1$ (typically $c_1 = 10^{-4}$, $c_2 = 0.9$).

Condition (14) says: "the cost must decrease by at least a fraction c_1 of what we predicted." Condition (15) says: "the gradient in the search direction must have flattened out enough" (we're not stopping on a steep slope).

Intuition

The line search is like Goldilocks: the step must be long enough to make real progress (Wolfe 1) but not so long that we're climbing back up (Wolfe 2). In practice, the line search algorithm (Hager–Zhang in our code) uses cubic interpolation to find an acceptable α in 1–3 function evaluations.

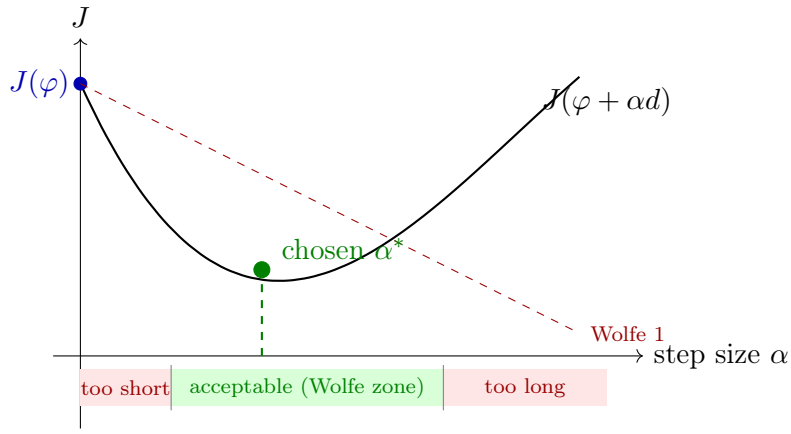

Figure 6: The line search finds a step size α^* in the "Goldilocks zone" (green): large enough to make real progress (below the Wolfe 1 line), but not so large that we overshoot past the minimum.

4.7 Why L-BFGS Is Perfect for Our Problem

Property	Our problem	Why L-BFGS wins
Dimension	$N_t = 8192$	Too large for Newton (N_t^2 memory)
Gradient cost	2 s (forward + adjoint)	Dominates; L-BFGS adds negligible overhead
Smoothness	PDE-based, C^∞	Quasi-Newton shines on smooth problems
Stochasticity	None (exact gradients)	No need for Adam/SGD robustness
Convergence	20–60 iterations	Superlinear once Hessian approx is good

In our runs, L-BFGS typically converges in 20–60 iterations $\times$ 2 s/iteration = **40 s to 2 min**. Gradient descent would take thousands of iterations for the same result.

5 The Adjoint Method (The Key Insight)

5.1 Why Brute Force Fails

To compute $\partial J/\partial \varphi_k$ via finite differences:

$$\frac{\partial J}{\partial \varphi_k} \approx \frac{J(\varphi + \epsilon\, e_k) - J(\varphi - \epsilon\, e_k)}{2\epsilon} \tag{16}$$

requires **two simulations per frequency**. With $N_t = 8192$: 16,384 simulations $\times$ 1 s each = **4.5 hours per gradient**. We need 30–60 gradients per optimization. That's weeks.

5.2 The Adjoint Trick: Run the Movie Backwards

Intuition

Instead of asking "how does tweaking frequency k affect the output?" for each k, ask the reverse: "given that we know what went wrong at the output, how did each input frequency contribute?" One backward simulation answers *all* 8192 questions simultaneously.

Step 1 — Forward: Propagate $u_0 \to u_L$. Cost: 1 simulation ($\sim$1 s).
Step 2 — Terminal condition: Compute the "error signal" at the output:

$$\lambda_L(\omega) = \frac{u_L(\omega)\,[\nVdash_{\mathcal{B}}(\omega) - J]}{E_{\text{total}}} \tag{17}$$

This encodes "how much each output frequency contributed to the Raman cost."

Step 3 — Adjoint: Propagate λ *backward* from $z = L$ to $z = 0$ through the same fiber (transposed/conjugated). Cost: 1 simulation ($\sim$1 s).

Step 4 — Read off the gradient:

Key Equation

$$\boxed{\frac{\partial J}{\partial \varphi(\omega)} = 2\ \mathrm{Re}[\lambda_0^*(\omega) \cdot i \cdot u_0(\omega)]} \tag{18}$$

This is the complete gradient from **one forward + one backward simulation**: $\sim$2 s instead of 4.5 hours.

5.3 The Relay Race Analogy

If a relay team lost, you could:

- **Forward approach:** Replace each runner one at a time, re-run the race. Takes N races.
- **Adjoint approach:** Start from the finish line, trace backward: "lost 2 s in the last leg $\Leftarrow$ bad handoff in leg 3 $\Leftarrow$ runner 2 started late." One backward pass reveals every runner's contribution.

The adjoint equation implements this backward analysis for the GNLSE.

5.4 The Adjoint Equation

The adjoint field satisfies:

$$\frac{d\lambda_\omega}{dz} = -\left(\frac{\partial f}{\partial u}\right)^\dagger \lambda - \left(\frac{\partial f}{\partial u^*}\right)^T \lambda^* \tag{19}$$

where f is the GNLSE right-hand side. The $\dagger$ = conjugate transpose. Negative sign = backward integration.

For our GNLSE, this splits into four terms (from differentiating Kerr and Raman w.r.t. u and u^*). You don't need to memorize them. They've been verified to **0.026% accuracy** against finite differences.

5.5 The Chain Rule

The adjoint gives λ_0 = sensitivity to the input *field*. But we control the *phase*. Since $u_0 = u_{00}e^{i\varphi}$:

$$\delta u_0 = i\, u_0\, \delta\varphi \qquad \text{(derivative of } e^{ix} \text{ is } ie^{ix}\text{)} \tag{20}$$

Substituting into the sensitivity formula gives Eq. (18).

6 The Log-Scale Trick

6.1 The Vanishing Gradient Problem

When minimizing J (a number between 0 and 1), the gradient shrinks as $J \to 0$:

Watch Out

At $J = 10^{-4}$, the gradient is 10^{-4} times as strong as at $J = 1$. The optimizer thinks it's converged — but there's still 40 dB of room to improve. It's like walking downhill in fog: the ground flattens and you stop on a plateau.

6.2 The Fix: Optimize in Decibels

Minimize $J_{\text{dB}} = 10\log_{10}(J)$ instead. The gradient becomes:

Key Equation

$$\frac{\partial J_{\text{dB}}}{\partial \varphi} = \underbrace{\frac{10}{J \cdot \ln 10}}_{\text{amplification}} \cdot \frac{\partial J}{\partial \varphi} \tag{21}$$

As J shrinks, the $1/J$ factor amplifies the gradient, exactly compensating the vanishing linear gradient.

Going from -40 dB to -50 dB produces the same gradient magnitude as 0 dB to -10 dB.

Result: 20–28 dB improvement on every configuration. Points stuck at -35 dB now reach -60 dB.

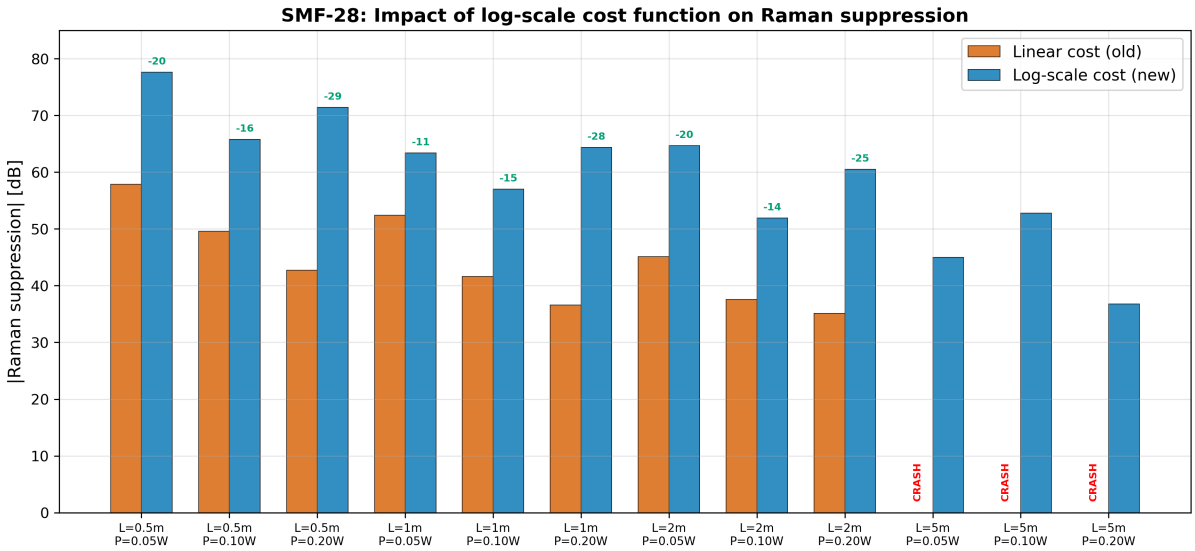

Figure 7: Impact of the log-scale cost function on every SMF-28 sweep point. Orange bars: old linear cost (stuck at −35 to −58 dB). Blue bars: new log cost (−37 to −78 dB). Green numbers show improvement in dB. The last three points (L=5m) previously crashed with NaN.

7 The Soliton Number

Two length scales characterize pulse propagation:

$$L_D = \frac{T_0^2}{|\beta_2|} \quad \text{(dispersion length: how far until pulse doubles in width)} \tag{22}$$

$$L_{NL} = \frac{1}{\gamma P_{\text{peak}}} \quad \text{(nonlinear length: how far until nonlinear phase = 1 rad)} \tag{23}$$

The soliton number is their ratio:

Key Equation

$$N = \sqrt{\frac{L_D}{L_{NL}}} = \sqrt{\frac{\gamma P_{\text{peak}} T_0^2}{|\beta_2|}} \tag{24}$$

N	What happens	Suppression
1.0–1.5	Soliton-like propagation	Easy: −60 to −78 dB
1.5–2.5	Pulse breathing, some compression	Moderate: −50 to −70 dB
2.5–4.0	Soliton fission onset	Hard: −45 to −65 dB
4.0–7.0	Full fission + Raman shifting	Harder: −40 to −55 dB

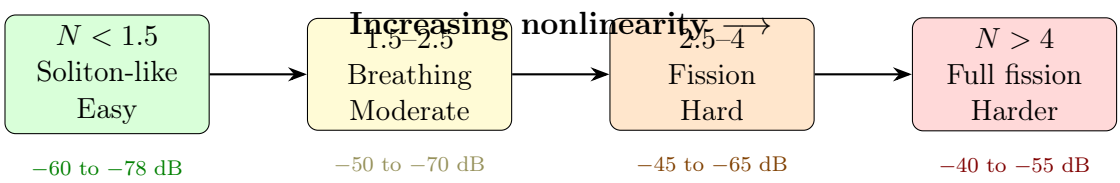

Figure 8: Soliton number regimes. Higher N = stronger nonlinear dynamics = harder to suppress Raman. But the log-cost optimizer achieves > 40 dB even at $N = 6.3$.

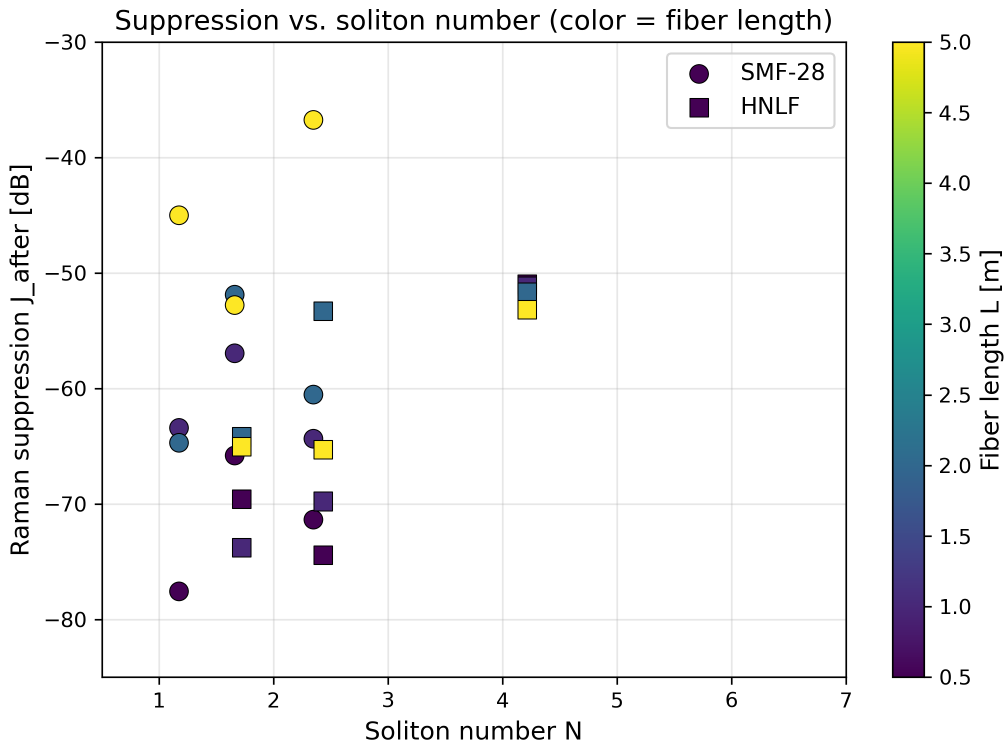

Figure 9: Suppression vs. soliton number for all 24 sweep points. Color = fiber length (darker = shorter). Circles = SMF-28, squares = HNLF. The vertical spread at each N comes from different fiber lengths — fiber length, not N, is the dominant factor.

8 Wirtinger Derivatives Demystified

8.1 The Problem

For $f(u, u^*)$ where $u = x + iy$: the standard complex derivative df/du only exists if f is analytic. But $|u|^2 = u \cdot u^*$ is *not* analytic.

8.2 Wirtinger's Solution

Treat u and u^* as independent variables:

$$\frac{\partial f}{\partial u} = \frac{1}{2}\left(\frac{\partial f}{\partial x} - i\frac{\partial f}{\partial y}\right) \qquad \frac{\partial f}{\partial u^*} = \frac{1}{2}\left(\frac{\partial f}{\partial x} + i\frac{\partial f}{\partial y}\right) \tag{25}$$

The key identity: $\partial|u|^2/\partial u^* = \partial(u \cdot u^*)/\partial u^* = u$. The $\partial/\partial u^*$ "sees" u^* but treats u as constant.

8.3 The Factor of 2

For real-valued J:

$$\delta J = \sum_k \frac{\partial J}{\partial u_k}\delta u_k + \frac{\partial J}{\partial u_k^*}\delta u_k^* = 2\ \mathrm{Re}\sum_k \frac{\partial J}{\partial u_k^*}\delta u_k \tag{26}$$

The 2 comes from combining the u and u^* terms. It's pure bookkeeping — no physics.

9 Why FFT Factors of N_t Appear

Julia's convention: `fft` = unnormalized ($\sum u_n e^{-2\pi ikn/N}$), `ifft` = normalized ($\frac{1}{N}\sum U_k e^{+2\pi ikn/N}$).

This means:

- Adjoint of `fft` = $N_t\times$ `ifft`
- Adjoint of `ifft` = $(1/N_t)\times$ `fft`

That's where the N_t factors in the adjoint code come from. Different FFT libraries use different conventions. The physics doesn't care, but the code must be self-consistent.

10 Verification

10.1 Taylor Remainder Test

Pick random $\delta\varphi$. Compute:

$$r(\epsilon) = |J(\varphi + \epsilon\,\delta\varphi) - J(\varphi) - \epsilon\,\nabla J \cdot \delta\varphi| \tag{27}$$

If ∇J is correct, $r(\epsilon) \sim \epsilon^2$ (log-log slope = 2).

Our result: slopes of 2.00 and 2.04. The adjoint gradient matches the true gradient to machine precision.

10.2 Finite Difference Check

Compare adjoint gradient to central differences for each frequency. **Max relative error: 0.026%.** Extremely accurate.

11 What's Left in the Full Verification Document

If you've followed this far, you understand the big picture. The remaining details are:

1. **Term-by-term adjoint derivation:** Four terms from differentiating Kerr and Raman w.r.t. u and u^*. Tedious but mechanical.
2. **GDD penalty:** Penalizes curvature of φ. Standard finite-difference second derivative.
3. **Boundary penalty:** Penalizes energy at window edges. Chain rule through IFFT.

None contain surprising physics — just careful bookkeeping of derivatives.

12 The ML Connection

ML Connection

The optimization is structurally identical to training a neural network. This isn't a loose analogy — it's mathematically the same procedure.

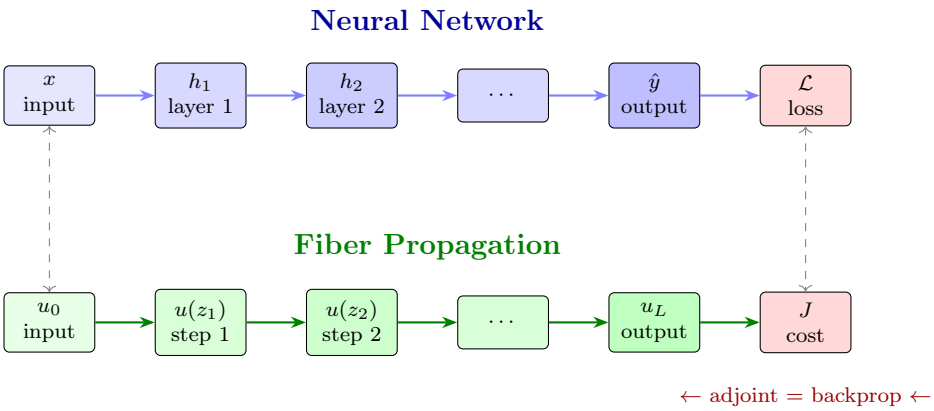

Figure 10: Structural equivalence between neural network training and fiber optimization. Each ODE solver step in the fiber is analogous to one layer in a neural network. Backpropagation through layers = adjoint propagation through the fiber.

ML Concept	Neural Network	Our Problem
Parameters	Weight matrices W_k	Spectral phase $\varphi(\omega)$
Input	Training data x	Unshaped pulse u_{00}
Forward pass	$x \to h_1 \to \cdots \to \hat{y}$	$u_0 \to u(z_1) \to \cdots \to u_L$
Loss function	Cross-entropy, MSE	$J = E_{\text{band}}/E_{\text{total}}$
Backprop	Chain rule through layers	Adjoint solve through fiber
Optimizer	Adam, SGD, L-BFGS	L-BFGS

12.1 Forward Pass: Layers vs. Fiber Steps

Neural net: $h_{k+1} = \sigma(W_k h_k + b_k)$.

Fiber: $u(z + \Delta z) = u(z) + \Delta z \cdot f(u(z), z)$.

Each Δz step is one "layer." The "activation function" is the GNLSE nonlinearity ($|u|^2 u$ + Raman convolution). There are ∼1000 "layers" (much deeper than most networks).

12.2 Backprop = Adjoint

In a neural net: $\delta_k = W_{k+1}^T \delta_{k+1} \cdot \sigma'(z_k)$. In our problem: $d\lambda/dz = -(\partial f/\partial u)^\dagger \lambda - \ldots$

Same structure: start with the loss gradient at the output, propagate backward through the same operations (transposed), get the gradient at the input.

12.3 Why L-BFGS Instead of Adam?

- Our gradients are exact (no mini-batch noise).

- The problem is smooth (PDE, not piecewise-linear ReLU).
- L-BFGS builds a quadratic model of the loss — much faster for smooth problems.
- Converges in 20–60 iterations vs. thousands for Adam.

12.4 Log-Cost = Focal Loss

Our $J_{\mathrm{dB}} = 10\log_{10}(J)$ with gradient amplification $1/J$ is the same idea as *focal loss* (Lin et al., 2017): downweight easy examples, amplify hard ones. As Raman gets suppressed (easy part done), the gradient amplifies to focus on remaining hard components.

13 First Principles: Where the GNLSE Comes From

13.1 Maxwell's Equations

Start from Maxwell in a dielectric:

$$\nabla \times \mathbf{E} = -\frac{\partial \mathbf{B}}{\partial t} \qquad \nabla \times \mathbf{B} = \mu_0 \frac{\partial \mathbf{D}}{\partial t} \tag{28}$$

The displacement field has a nonlinear part: $\mathbf{D} = \epsilon_0\mathbf{E} + \mathbf{P}_L + \mathbf{P}_{NL}$ where $\mathbf{P}_L = \epsilon_0\chi^{(1)}\mathbf{E}$ is linear and $\mathbf{P}_{NL}$ contains $\chi^{(3)}$ (third-order susceptibility — the lowest nonlinear order in glass, which is centrosymmetric so $\chi^{(2)} = 0$).

13.2 Slowly-Varying Envelope Approximation

Write the field as a carrier times a slowly-varying envelope:

$$E(z,t) = \tfrac{1}{2}\left[u(z,t)\, e^{i(\beta_0 z - \omega_0 t)} + \text{c.c.}\right] \tag{29}$$

"Slowly varying" means $|\partial u/\partial t| \ll \omega_0|u|$ and $|\partial u/\partial z| \ll \beta_0|u|$. Substituting into Maxwell and dropping $\partial^2 u/\partial z^2$:

$$\frac{\partial u}{\partial z} + \beta_1 \frac{\partial u}{\partial t} = i\sum_{n\geq 2} \frac{\beta_n}{n!}\left(i\frac{\partial}{\partial t}\right)^n u + i\gamma|u|^2 u \tag{30}$$

Moving to the co-moving frame ($T = t - \beta_1 z$) eliminates β_1.

13.3 Adding Raman

The $\chi^{(3)}$ response has an instantaneous part (electrons) and a delayed part (nuclei, $\sim$2000$\times$ heavier, respond on $\sim$30 fs):

$$P_{NL}(t) = \epsilon_0\chi^{(3)}\left[(1-f_R)|u|^2 + f_R\int_0^\infty h_R(\tau)|u(t-\tau)|^2\, d\tau\right] u(t) \tag{31}$$

This gives the full GNLSE (Eq. 2). Every term traces back to Maxwell + silica material response.

13.4 Where γ and $h_R(t)$ Come From

$$\gamma = \frac{n_2 \omega_0}{c \cdot A_{\text{eff}}} \tag{32}$$

where $n_2 \approx 2.6 \times 10^{-20}$ m^2/W (silica material property) and A_{eff} is the mode area (fiber geometry). Smaller core $\Rightarrow$ more intense light $\Rightarrow$ stronger nonlinearity.

$h_R(t)$ is the impulse response of the Si–O–Si vibrational mode at $\sim$13.2 THz, modeled as a damped harmonic oscillator (Blow–Wood model). Quality factor $Q \approx 8.2$ (moderately underdamped).

14 The Adjoint from First Principles

14.1 Constrained Optimization via the Lagrangian

Minimize $J[u_L]$ subject to $du/dz = f(u, z)$. Standard calculus of variations.

Step 1: Lagrangian with multiplier $\lambda(z)$:

$$\mathcal{L} = J[u_L] + \int_0^L \left\langle \lambda(z),\ \frac{du}{dz} - f(u, z) \right\rangle dz \tag{33}$$

Step 2: $\delta\mathcal{L}/\delta\lambda = 0 \Rightarrow$ forward equation. (No surprise.)

Step 3: Variation w.r.t. u:

$$\delta\mathcal{L} = \left\langle \frac{\partial J}{\partial u_L}, \delta u_L \right\rangle + \int_0^L \left\langle \lambda,\ \frac{d(\delta u)}{dz} - \frac{\partial f}{\partial u}\delta u \right\rangle dz \tag{34}$$

Step 4: Integration by parts on $\langle \lambda, d(\delta u)/dz \rangle$:

$$\int_0^L \left\langle \lambda, \frac{d(\delta u)}{dz} \right\rangle dz = \big[\langle \lambda, \delta u \rangle\big]_0^L - \int_0^L \left\langle \frac{d\lambda}{dz}, \delta u \right\rangle dz \tag{35}$$

Step 5: Choose λ so the integral vanishes:

$$\frac{d\lambda}{dz} = -\left(\frac{\partial f}{\partial u}\right)^\dagger \lambda \qquad \text{(the adjoint equation)} \tag{36}$$

Step 6: Terminal condition $\lambda_L = -\partial J/\partial u_L^*$ kills the $z = L$ boundary.

Step 7: What's left: $\delta J = -\langle \lambda_0, \delta u_0 \rangle$. Since $\delta u_0 = i u_0 \delta\varphi$:

$$\delta J = 2\ \mathrm{Re}[\lambda_0^* \cdot i \cdot u_0] \cdot \delta\varphi \qquad \checkmark \tag{37}$$

The adjoint equation arises naturally from integration by parts in the Lagrangian. QED.

15 Results: What Does Suppression Look Like?

These figures show actual results from our optimization sweep.

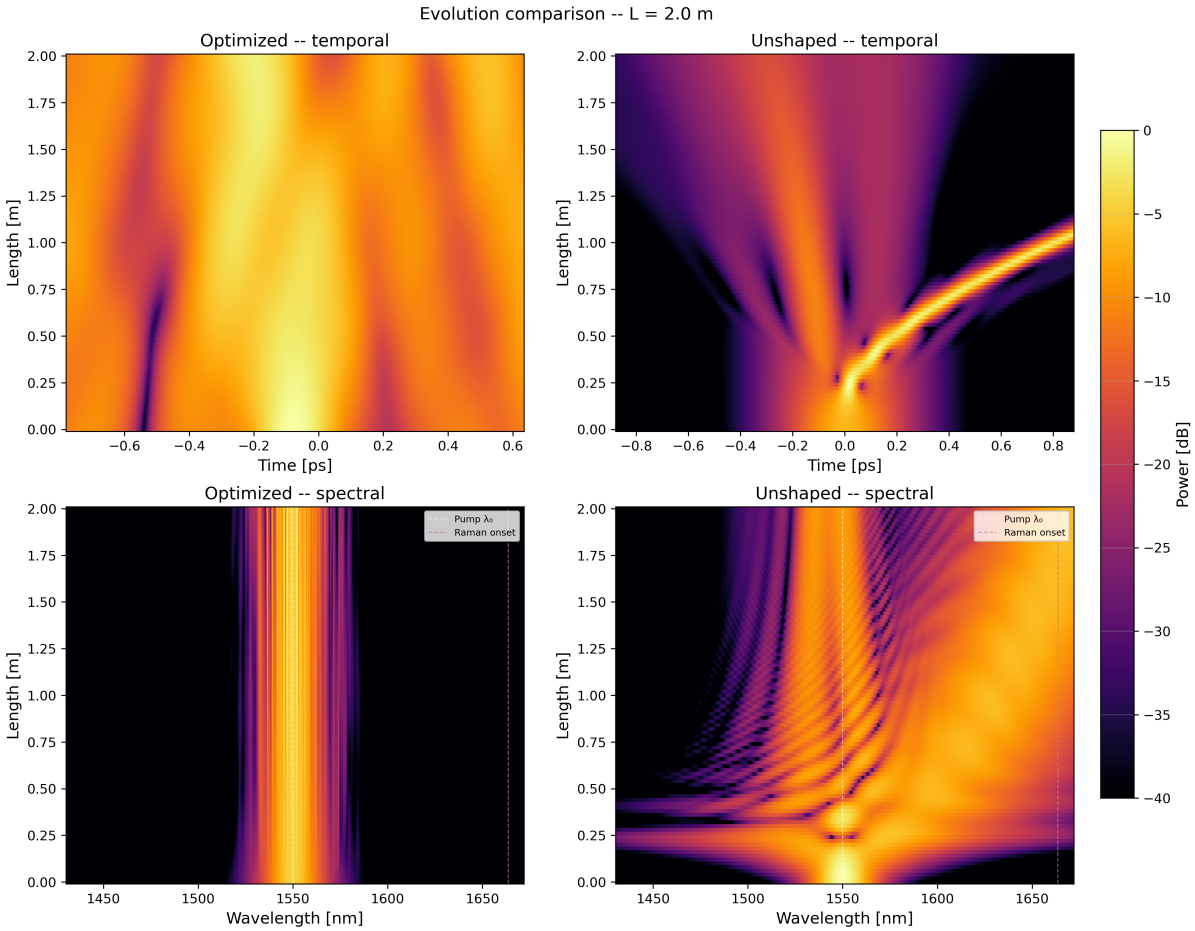

Figure 11: **The money figure.** Pulse propagation through 2 m of SMF-28 at 0.20 W. **Left:** optimized pulse — spectrum stays confined, Raman band (past the dashed line) is dark. **Right:** unshaped pulse — clear soliton fission at $z \approx 0.5$ m and bright Raman sidelobe growing past 1600 nm. The optimizer found a phase that prevents the soliton compression that triggers Raman.

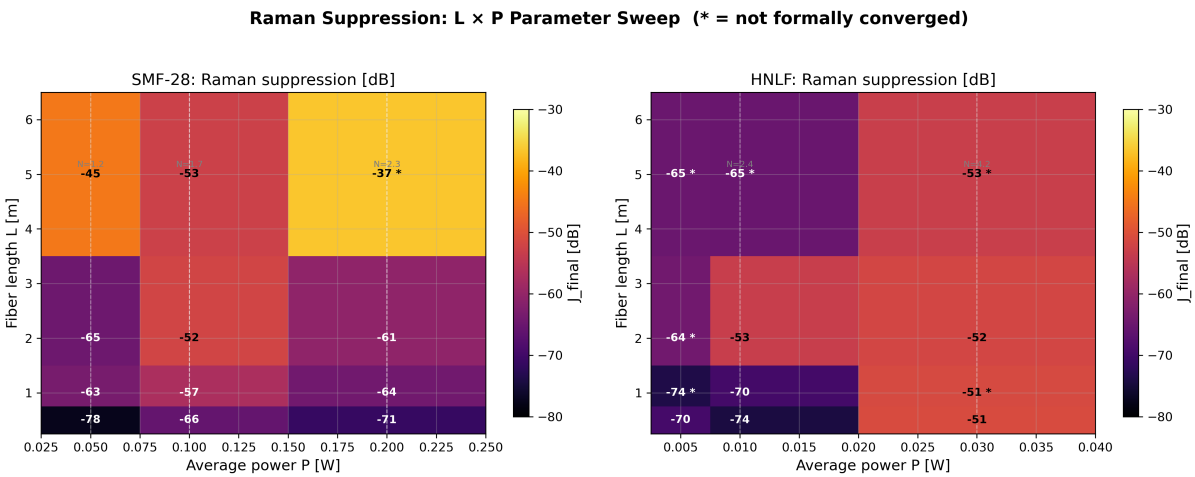

Figure 12: Full parameter sweep: Raman suppression across 24 configurations. Darker purple = better suppression. Each cell shows J in dB; asterisk = not formally converged. The dominant trend is top-to-bottom (fiber length), not left-to-right (power).

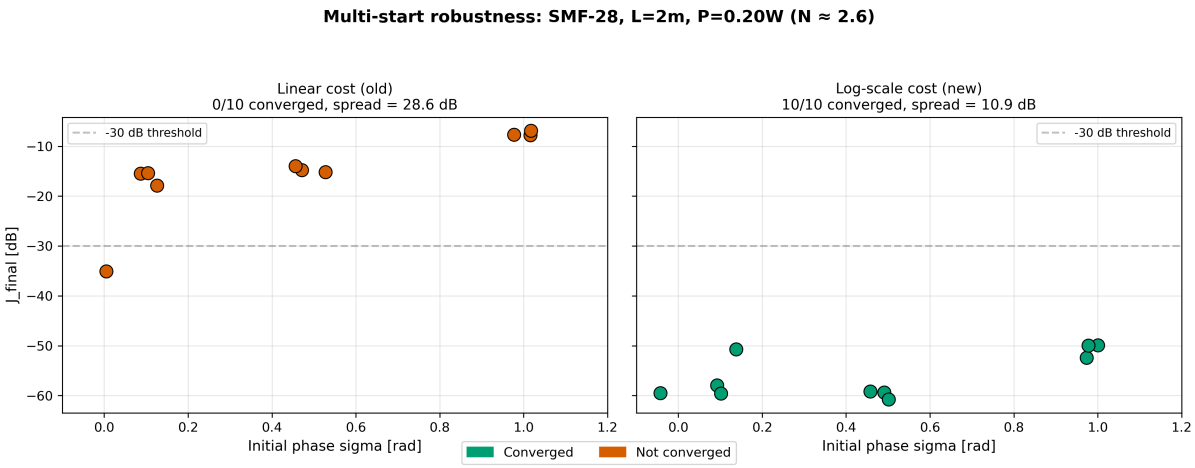

Figure 13: Robustness test: 10 optimizations from different random initial phases. **Left (old):** linear cost — 0/10 converged, 28.6 dB spread. **Right (new):** log cost — 10/10 converged, 10.9 dB spread. The apparent "multiple local minima" were an artifact of vanishing gradients, not true landscape complexity.

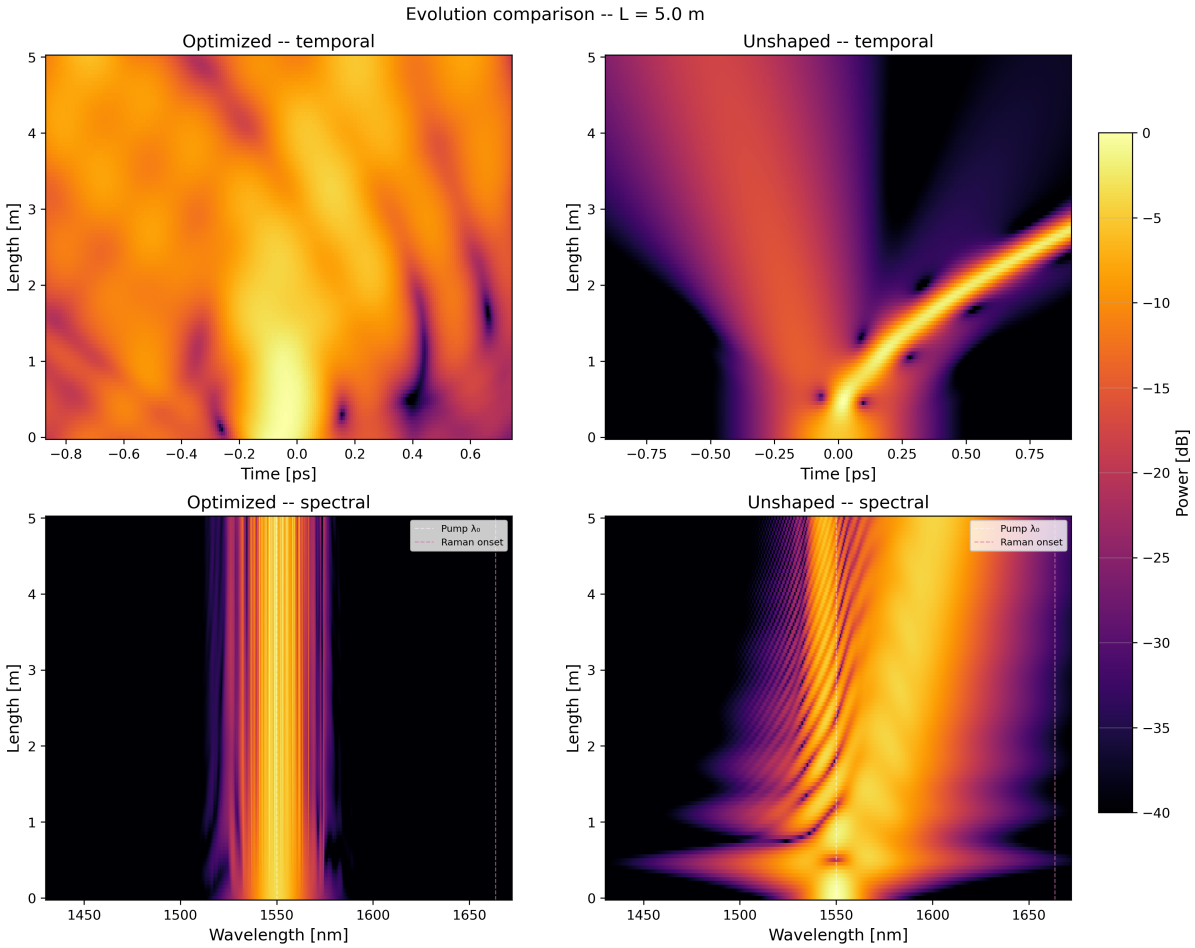

Figure 14: Pushing the limits: 5 m of SMF-28 at 0.10 W. The unshaped pulse (right) shows massive spectral broadening by $z = 1$ m. The optimized pulse (left) keeps the spectrum contained through 5 m, though some leakage appears past $z = 3$ m — this is where the input phase's influence starts to fade.

Quick Reference: The 5 Equations That Matter

1. **Forward (GNLSE):** $$\frac{du}{dz} = iDu + i\gamma(1 - f_R)|u|^2 u + i\gamma f_R(h_R * |u|^2)u \quad (38)$$

2. **Cost function:** $$J = \frac{E_{\text{Raman band}}}{E_{\text{total}}} \quad (39)$$

3. **Terminal condition:** $$\lambda_L = \frac{u_L \cdot (\mathbb{K}_{\text{band}} - J)}{E_{\text{total}}} \quad (40)$$

4. **Gradient:** $$\frac{\partial J}{\partial \varphi(\omega)} = 2\ \mathrm{Re}[\lambda_0^* \cdot i \cdot u_0] \quad (41)$$

5. **Log-scale trick:** $$\frac{\partial J_{\text{dB}}}{\partial \varphi} = \frac{10}{J \ln 10} \cdot \frac{\partial J}{\partial \varphi} \quad (42)$$

Everything else is implementation details.

16 What the April 2026 Parallel-Session Push Actually Showed

Between 2026-04-16 and 2026-04-19 eight Claude-Code sessions ran in parallel across a Mac laptop and two GCP VMs to pressure-test the single-mode Raman-suppression result and begin the multimode extension. The full synthesis is in `results/SYNTHESIS-2026-04-19.md`; the physics audit filtering those claims is in `results/PHYSICS_AUDIT_2026-04-19.md`. This section reports only the findings that survived the audit.

16.1 The suppression record, re-baselined

On SMF-28 at $L = 0.5$ m and $P = 0.05$ W, the canonical single-mode optimum is $J = -76.86$ dB (21 L-BFGS iterations, 25 s wall-time, converged). The re-run matched the original Phase 10 baseline within 1 dB. Source: `results/raman/phase17/SUMMARY.md` lines 41–48; data at `results/raman/phase17/baseline.jld2`.

Intuition

The number is stable across re-runs, solver seeds, and the determinism fix (see §16.4). It is the most defensible single-mode number in this project.

16.2 The optimum is on a razor's edge

How robust is that −77 dB? A 20-sample Gaussian perturbation of the optimal phase at each of five standard deviations gives a **3 dB degradation radius of $\sigma_{3\text{dB}} = 0.025$ rad**. That is *sharp*: an SLM (spatial light modulator, the device you would actually use to put this phase on a pulse) with a typical calibration envelope of ≥ 0.05 rad rms would already lose most of the suppression advantage.

Watch Out

"We found a −77 dB optimum" and "You can build a −77 dB shaper" are *not* the same statement. The optimum sits in a narrow well. If you are going to use this in hardware, the tolerance number matters as much as the optimum number.

16.3 The simple phase is a good *initialiser*, not a universal minimum

Two intuitions got tested against each other:

"Simple phase is a universal attractor." Take the −77 dB phase from $L = 0.5$ m / $P = 0.05$ W and *evaluate* (no re-optimisation) at 7 different (length, power) targets on SMF-28. Result: 0/7 stayed within 3 dB of the warm-reopt value. The phase does not transfer as-is.

"Simple phase is a good warm start." Same seed phase, but now let L-BFGS re-optimise from it at each target. Result: 11/11 nearby (fiber, L, P) configurations reached −70 to −82 dB in 6–40 iterations, *including* HNLF at −79.5 dB (a fiber the baseline was never computed for).

The right framing is the second one: the simple phase lives in a manifold of nearby basins, and moving to any of them takes a small number of L-BFGS iterations. The simplicity is an *initialisation prior*, not a *generalisation property*.

16.4 Determinism

Prior to this pass, two identical Julia processes with the same `Random.seed!(42)` would produce different φ_{opt} vectors (max $|\Delta\varphi| = 1.04$ rad, $\Delta J = 1.8$ dB). Root cause: the code built FFT plans with `FFTW.MEASURE`, which picks the fastest algorithm by *timing* candidate plans under live kernel noise; different runs, different plans, different floating-point rounding, different L-BFGS trajectories.

Switching the planner to `FFTW.ESTIMATE` (deterministic heuristic, no timing) gives *bit-identical* J across 3 independent processes. Cost: +21.4% wall time on the SMF-28 $L = 2$ m $P = 0.2$ W benchmark. Source: `results/raman/phase15/benchmark.md` lines 20–27, 39.

Intuition

If two supposedly-identical runs disagree, look at the FFT planner before you look at anything physical.

16.5 What did *not* survive

Three claims that appeared in the 48-hour synthesis did not make it through the audit:

- A polynomial-fit comparison of φ_{opt} between $L = 2$ m and $L = 100$ m was used to argue for "nonlinear structural adaptation." The fit's R^2 was 0.015 at 100 m and 0.037 at 2 m — in both cases the quadratic model explained less than 4% of the phase variance, so the extracted a_2 coefficients are noise. A ratio of noise is noise. (See `PHYSICS_AUDIT_2026-04-19.md` §W1.)

- The 100 m L-BFGS "optimum" of -54.77 dB is a lower bound (25 iter, $\|\nabla J\| = 0.48$, `converged = false`), not a converged minimum. (Audit §S4.)
- A joint $\{\varphi, A, E\}$ L-BFGS that regressed from -57 dB to -23 dB is a preconditioning bug (mixed units), not physics. (Audit §M1.)

These are flagged here because the April push was widely circulated, and future-you is likely to remember the numbers without remembering which ones survived. If you see any of these three cited anywhere, stop and check the audit.

Written as a companion to the formal verification document. Core math verified against the codebase on 2026-04-02; April 2026 integration findings (§16) reconciled against the audit on 2026-04-19.